WIF3009
Python for Scientific Computing

Data Importing and Cleaning

Data Importing

- Python can read numerous file types. Common file types for Python are CSV, TXT and tables from relational databases.
- In data science, data are often stored in external files. As such, data importing is a critical skill that you must acquire.
- You can import data into Python through various ways, depending on the file type. You may need external packages like NumPy, Pandas, sqlalchemy, pymsql.

Data Importing (flat files)

- A flat file is when data is contained only within a single file. It does not contain any relations like a relational database does.
- A text file (.txt) is an example of flat file. To import, you can use Python's `open` and `read` functions.

```
[399]: reader = open('huck_fin.txt','r')

[400]: reader.read()

[400]: 'Along during the morning I borrowed a sheet ar
       put them in it, and we went down and got the fo
       what pap always called it; but Tom said it warr
       and prisoners don't care how they get a thing s
       o crime in a prisoner to steal the thing he nee
       was representing a prisoner, we had a perfect r
       ourselves out of prison with. He said if we war
       an, ornery person would steal when he warn't a
       handy. And yet he made a mighty fuss, one day,
       it; and he made me go and give the niggers a di
       was, we could steal anything we needed. Well, I
       ut of prison with; there's where the difference
       o Jim to kill the seneskal with, it would a bee
       ge in my representing a prisoner if I got to se
       time I see a chance to hog a watermelon. '

[401]: reader.close()
```

Data Importing (flat files)

- `file.read()` returns a single string for the entire file.
- Use `file.readline()` to read a single line from the file or `file.readlines()` to read each line as a separate string.
- Another Pythonic way of reading file is:

```
[403]: with open('huck_fin.txt','r') as reader:
           for line in reader.readlines():
               print(line)

Along during the morning I borrowed a sheet a
ut them in it, and we went down and got the f
hat pap always called it; but Tom said it war
and prisoners don't care how they get a thing
o crime in a prisoner to steal the thing he r
was representing a prisoner, we had a perfect
ourselves out of prison with. He said if we w
an, ornery person would steal when he warn't
handy. And yet he made a mighty fuss, one day
it; and he made me go and give the niggers a
was, we could steal anything we needed. Well,
ut of prison with; there's where the differen
o Jim to kill the seneskal with, it would a b
ge in my representing a prisoner if I got to
time I see a chance to hog a watermelon.
```

Data Importing (flat files)

- The context manager `with` ensures that Python closes the file connection when it has finished reading/writing.
- It saves you from the hassle of writing the `file.close()` statement.
- Another alternative of reading flat files is by using external packages like NumPy.
- NumPy provides `loadtxt()` function to read text files containing numerics.

Data Importing (Pandas)

- What is Pandas?
- Is it this guy ----------→

Data Importing (Pandas)

- **pandas** is a fast, powerful, flexible and easy to use open-source data analysis and manipulation tool, built on top of the Python programming language.
- A fast and efficient **DataFrame** object for data manipulation with integrated indexing;
- Tools for **reading and writing data** between in-memory data structures and different formats: CSV and text files, Microsoft Excel, SQL databases, and the fast HDF5 format

Data Importing (Pandas)

- How to get Pandas? Easy.
- Just run `***pip install pandas***` in the terminal or `***!pip install pandas***` in the Jupyter Lab notebook.

```
[404]: !pip install pandas

       Requirement already satisfied: pandas in ./
       Requirement already satisfied: numpy<2,>=1.
       as) (1.26.3)
       Requirement already satisfied: python-dateu
       m pandas) (2.8.2)
       Requirement already satisfied: pytz>=2020.1
       (2023.3.post1)
       Requirement already satisfied: tzdata>=2022
       s) (2023.4)
       Requirement already satisfied: six>=1.5 in
       til>=2.8.2->pandas) (1.16.0)
```

Data Importing (CSV)

- What is CSV?
  Comma Separated Values.
- How to use pandas?
  Import it first.
- Example shows how to read a CSV file.

```
[409]: import pandas as pd

       mydata  = pd.read_csv('username.csv')
       mydata
```

[409]:

	Username	Identifier	First name	Last name
0	booker12	9012	Rachel	Booker
1	grey07	2070	Laura	Grey
2	johnson81	4081	Craig	Johnson
3	jenkins46	9346	Mary	Jenkins
4	smith79	5079	Jamie	Smith

Data Importing (CSV)

- What if the delimiter is not comma (,).
- If the delimiter is something else...use the delimiter parameter.
- `read_csv()` can accept other arguments such as `delimiter`, `header`, `nrows`, `skiprows` etc.

```
[407]: import pandas as pd

       mydata  = pd.read_csv('username.csv',delimiter=';')
       mydata
```

[407]:

	Username	Identifier	First name	Last name
0	booker12	9012	Rachel	Booker
1	grey07	2070	Laura	Grey
2	johnson81	4081	Craig	Johnson
3	jenkins46	9346	Mary	Jenkins
4	smith79	5079	Jamie	Smith

Data Importing (Excel file)

- You can also import Excel file, and access a specific sheet.
- All just using Pandas.

```
[420]: import pandas as pd

       mydata  = pd.ExcelFile('username.xlsx')
       mydata.sheet_names

[420]: ['sheet1', 'sheet2']
```

```
[424]: df1 = mydata.parse('sheet1')
       df1
```

[424]:

	Username	Identifier	First name	Last name
0	booker12	9012	Rachel	Booker
1	grey07	2070	Laura	Grey
2	johnson81	4081	Craig	Johnson
3	jenkins46	9346	Mary	Jenkins
4	smith79	5079	Jamie	Smith

```
[425]: df2 = mydata.parse('sheet2')
       df2
```

[425]:

	Username	Identifier	First name	Last name
0	mfa23	124	James	Grant
1	oak36	6346	Konrad	Litt
2	zxc23	1234	Hendy	Johnson
3	polly98	74342	Lilo	Saz
4	zxc88	6335	John	Poll

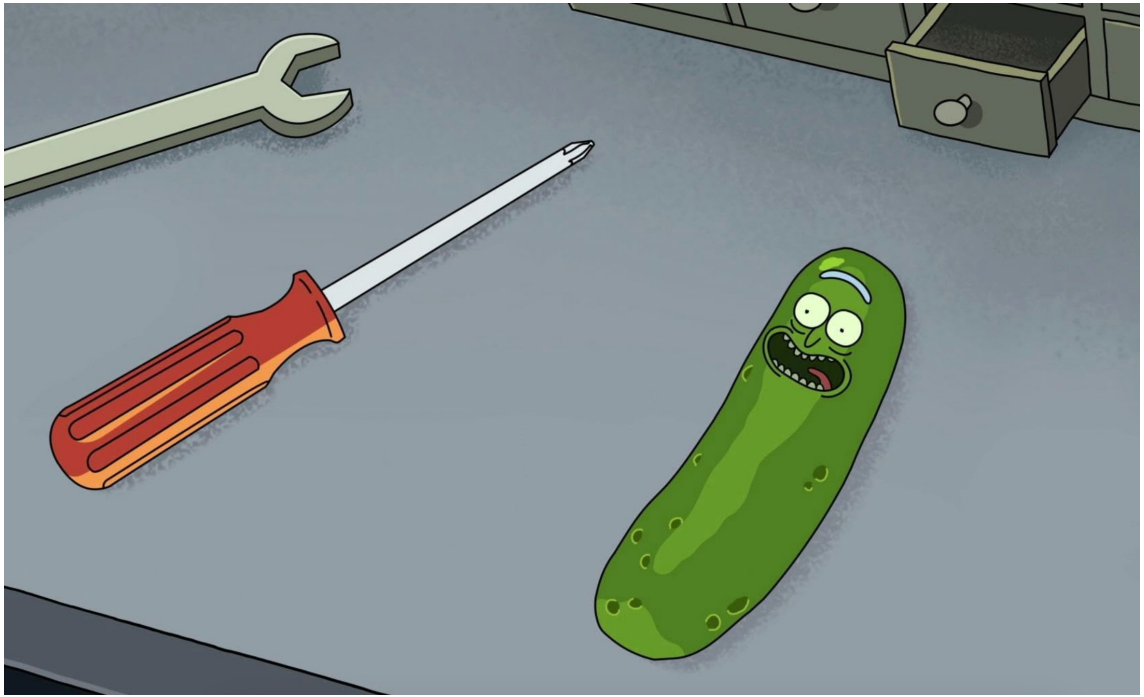
Data Importing (Pickle)

- Pickled files are native file type to Python.
- There are many data types for which it is not obvious how to store them. For example, dictionaries and machine learning models. Should we store them in .txt? .csv?
- Using pickle, these files are serialized. In simpler terms, these files are converted from object into bytestream (machine readable).

NOT THIS GUY →

Data Importing (Pickle)

- And definitely not this guy too....



Data Importing (Pickle)

- How to pickle a file:

```
import pickle

heights_dict = {'Abu':1.24, 'Wong':1.67, 'Bryan':1.98}

with open('dictionary.pkl','wb') as f:
    pickle.dump(heights_dict, f)
```

- How to unpickle/read a pickle file:

```
[433]: import pickle

       with open('dictionary.pkl','rb') as f:
           new_dict = pickle.load(f)

       new_dict

[433]: {'Abu': 1.24, 'Wong': 1.67, 'Bryan': 1.98}
```

Data Importing (JSON)

- Big data sets are often stored, or extracted as JSON.
- JSON is plain text, but has the format of an object, and is well known in the world of programming, including Pandas.
- In our examples we will be using a JSON file called 'data.json'.

[472]:
```
from io import StringIO
import pandas as pd

with open('data.json','r') as f:
    df = pd.read_json(StringIO(f.read()))

df
```

[472]:

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
...	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

169 rows × 4 columns

Data Exporting

- If you can import various files into Python, you can also export various files outside Python!
- Python has rich exporting methods. Most of them are available through Pandas library.
- For example:
  - `dataframe.to_csv('exported.csv')`
  - `writer = pd.ExcelWriter('exported.xlsx')`
  - `dataframe.to_excel(writer, 'Sheet1')`
  - `dataframe.to_json('exported.json')`
  - `dataframe.to_pickle('exported.pkl')`

Data Cleaning

- Data almost never comes in clean.
- It highly likely that the data that comes into your possession will contain numerous problems that require some cleaning before it is ready for analysis.
- Data cleaning is the most crucial part in data science.
- As the famous quote goes, “Garbage in, garbage out”. In other words, if you do not put proper attention in cleaning your data, you might risk having low quality output/results.
- The first step in cleaning your data is to diagnose your data for problems: **exploratory data analysis (EDA)**

Data Cleaning (common data problems)

- Inconsistent column names
- Missing data
- Outliers
- Duplicate rows
- Untidy
- Need to reshape/process columns
- Wrong data types

Data Cleaning (EDA)

- The figure shows a sample dataset containing unclean data.
- The column names are inconsistent:
  - first letter capitalization.
- There is a missing data.
- The country names are stored in French.

```
In [53]: data = pd.ExcelFile('population_literacy.xlsx')

In [54]: df = data.parse('Sheet1')

In [55]: df.head()

Out[55]:
```

	Country	Continent	female literacy	fertility	population
0	Chine	ASI	90.5	1.769	1.324655e+09
1	Inde	ASI	50.8	2.682	1.139965e+09
2	USA	NAM	99.0	2.077	3.040600e+08
3	Indonésie	ASI	88.8	2.132	2.273451e+08
4	Brésil	LAT	90.2	1.827	NaN

Data Cleaning (EDA)

- You can visually inspect your data in a data frame by using the `head()` method.
- The `head()` method by default returns the first 5 rows of the data frame. You can specify the number of rows to be displayed e.g. `df.head(10)`

```
In [53]: data = pd.ExcelFile('population_literacy.xlsx')

In [54]: df = data.parse('Sheet1')

In [55]: df.head()

Out[55]:
```

	Country	Continent	female literacy	fertility	population
0	Chine	ASI	90.5	1.769	1.324655e+09
1	Inde	ASI	50.8	2.682	1.139965e+09
2	USA	NAM	99.0	2.077	3.040600e+08
3	Indonésie	ASI	88.8	2.132	2.273451e+08
4	Brésil	LAT	90.2	1.827	NaN

Data Cleaning (EDA)

- Use the `columns` and `shape` attribute to check the column names and data frame shape. Notice that there is a white space in the 'Country ' and *'Continent* ' column names.
- The `info()` method gives a quick overview of the data frame. Notice that the *population* column contains less values compared to other columns.

```
In [65]: df.columns

Out[65]: Index(['Country ', 'Continent  ', 'female literacy', 'fertility',
                 'population'],
                dtype='object')

In [66]: df.shape

Out[66]: (162, 5)

In [67]: df.info()

         <class 'pandas.core.frame.DataFrame'>
         RangeIndex: 162 entries, 0 to 161
         Data columns (total 5 columns):
         Country            162 non-null object
         Continent          162 non-null object
         female literacy    162 non-null float64
         fertility          162 non-null float64
         population         147 non-null float64
         dtypes: float64(3), object(2)
         memory usage: 6.4+ KB
```

Data Cleaning (EDA)

- You can also perform frequency counts on the columns to observe the distribution.
- To calculate the frequency, you can use the `value_counts()` method.
- This method can accept a Boolean argument `dropna`. By default, it is set to True. If set to False, all rows including missing data are counted.
- Notice the data type. Should the continent be an integer?

```
In [72]: df.Continent.value_counts(dropna=False)

Out[72]: AF      49
         ASI     47
         EUR     36
         LAT     24
         OCE      4
         NAM      2
         Name: Continent, dtype: int64

In [73]: df['Continent'].value_counts(dropna=False)

Out[73]: AF      49
         ASI     47
         EUR     36
         LAT     24
         OCE      4
         NAM      2
         Name: Continent, dtype: int64
```

Importing other file types.pdf

Two ways of accessing the columns: using dot (.) or using square brackets [ ].

Data Cleaning (EDA)

- Since the sample dataset is about world population, you should expect that each country in the world appears only once in the dataset.
- You can explore if your dataset contains duplicate by the following.
- It seems Denmark appeared twice in the dataset.

```
In [84]: df['Country'].value_counts(dropna=False).head()

Out[84]: Denmark                                2
         Samoa                                  1
         Thaïlande                              1
         Singapour                              1
         L'ex-Rép. yougoslave de Macédoine      1
         Name: Country, dtype: int64
```

Data Cleaning (EDA)

- Based on our `info()` output in earlier slide, we noticed that the number of entries in the population column was lower compared to other columns.
- We can perform a value count to confirm this.
- Notice the number of NaNs in the column.

```
In [86]: df.population.value_counts(dropna=False).head()

Out[86]: NaN            15
          5493621.0      2
          46258200.0     1
          31656865.0     1
          10622413.0     1
         Name: population, dtype: int64
```

Data Cleaning (EDA)

- Besides that, you can also perform summary statistics such as mean, median and standard deviation on numerical data.
- These statistics can prove to be crucial as you can detect any anomalies in the data.
- You can use the describe() method on your data frame. This method returns the summary statistical values of all columns. The values include count, min, max, standard deviation and quartile values

Data Cleaning (EDA)

- For example, the maximum value in the population column based on the figure is more than 3 billion. It is an anomaly as no country in the world has a population more than 3 billion currently.

```
In [90]: df.describe()
```

Out[90]:

	female literacy	fertility	population
count	163.000000	163.000000	1.480000e+02
mean	80.223313	2.872356	6.360217e+07
std	23.028749	1.425467	3.079884e+08
min	12.600000	0.966000	1.035660e+05
25%	66.550000	1.824500	3.574732e+06
50%	90.200000	2.362000	9.776874e+06
75%	98.500000	3.877500	2.881632e+07
max	100.000000	7.069000	3.336134e+09

Data Cleaning (Visual EDA)

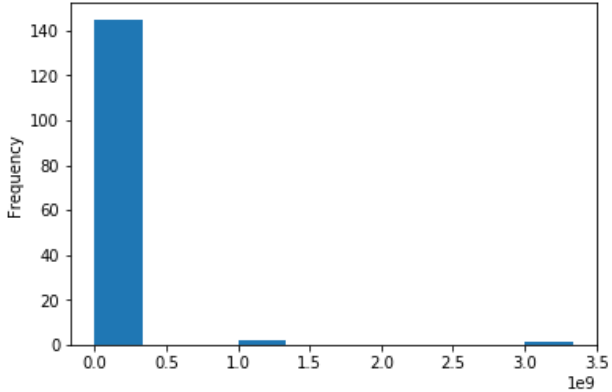
- Apart from exploring your data using functions described previously, you can also perform visual EDA.
- Visual EDA is useful to spot outliers and obvious errors.
- To effectively perform visual EDA, you need to understand the types of plots (visualization) that suit your data.
- Example of visualizations are histogram, column chart, scatter chart and line chart.
- You can use the Pandas `plot()` method on your data frame and the `matplotlib` package to display the plot.

Data Cleaning (Visual EDA)

- **Histograms**
  - Histogram is used to plot frequencies of continuous data.
  - The main objective of this plot is to observe the frequency distribution across data.
  - Observe that most countries' population lie in the range 0.0 to 0.4 billion while there are outliers 1.0 – 1.4 billion (China and India) and in the 3 billion range (error).

```
In [4]: df.population.plot('hist')

Out[4]: <matplotlib.axes._subplots.AxesSubplot at 0x110e485f8>
```

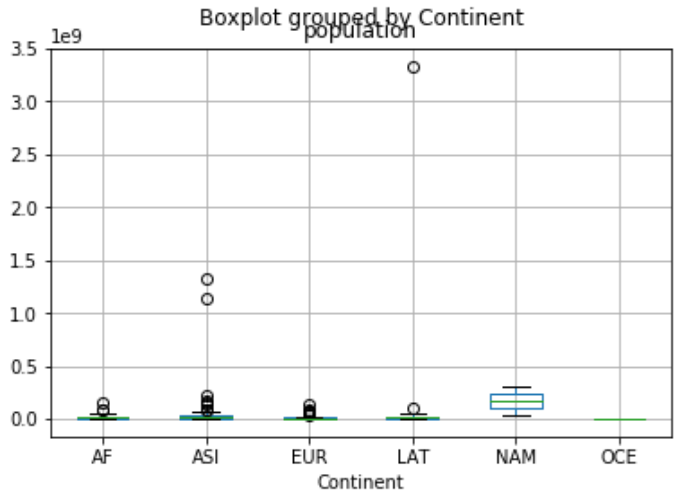


Data Cleaning (Visual EDA)

- **Boxplot**
  - It is useful to detect outliers.
  - It tells us the min, max, quartile values (25th, 50th, 75th percentiles).
  - Observe the two outliers in the ASI continent and one outlier in LAT continent.

```
In [13]: df.boxplot(column='population', by='Continent')

Out[13]: <matplotlib.axes._subplots.AxesSubplot at 0x111823e10>
```

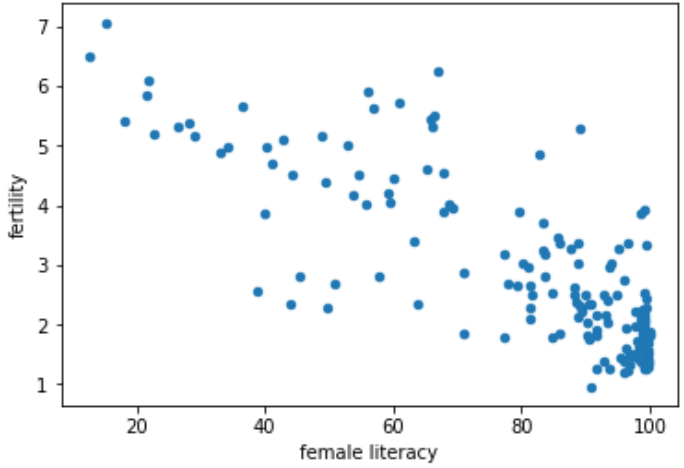


Data Cleaning (Visual EDA)

- **Scatter plot**
  - It is used to observe the relationship/correlation between two variables.
  - Can also detect outliers.
  - Based on the figure, we notice the negative correlation between female literacy and fertility across the world.

```
In [18]: df.plot.scatter(x='female literacy', y='fertility', )

Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x111cd4278>
```



Data Cleaning

Tidy Data

- You should store data in a proper and meaningful structure.
- You should have a standard way of organizing your data in a dataset.
- This will help in ensuring smooth data analysis process.
- Principles of tidy data are as follows:
  - Columns represent separate variables.
  - Rows represent individual observation/entry.
  - The collection of rows and columns form a table.

Data Cleaning

Tidy Data

	name	treatment a	treatment b
0	Daniel	-	42
1	John	12	31
2	Jane	24	27

	0	1	2
name	Daniel	John	Jane
treatment a	-	12	24
treatment b	42	31	27

- The two tables above hold a few issues:
  - Table on the left has two columns holding values: type of treatments.
  - Table on the right has rows holding the variable names.

Data Cleaning

Tidy Data

- There are two considerations when tidying up your data:
  - Better for reporting
  - Better for analysis
- These two considerations may have different data structure.
- The initial table on the right may be suitable for reporting. However, the transformed table is suited for easier analysis.

	name	treatment a	treatment b
0	Daniel	-	42
1	John	12	31
2	Jane	24	27

	name	treatment	value
0	Daniel	treatment a	-
1	John	treatment a	12
2	Jane	treatment a	24
3	Daniel	treatment b	42
4	John	treatment b	31
5	Jane	treatment b	27

Data Cleaning

Tidy Data

- You can tidy up your data using Pandas `melt()` method.
- Melting a data frame will change the shape of the data frame from wide to long.
- In other words, it reduces the number of columns and increases the number of rows.

```
In [29]: pd.melt(frame=df, id_vars='name', value_vars=['treatment a', 'treatment b'],
         var_name='treatment', value_name='reading')
```

Out[29]:

	name	treatment	reading
0	Aisyah	treatment a	20
1	Fatimah	treatment a	16
2	Melur	treatment a	10
3	Aisyah	treatment b	25
4	Fatimah	treatment b	31
5	Melur	treatment b	7

Data Cleaning

Tidy Data

- The `melt()` method melts the columns defined in the `value_vars` argument leaves out the column in `id_vars` argument. If `melt()` is passed without any of these variables, this method will melt all the columns in the data frame.
- By default, melted columns will be renamed to ‘variable’ and ’value’. You can customize these names through the `var_name` and `value_name` arguments.

Data Cleaning

Tidy Data

- The opposite of `melt()` is the `pivot()` method.
- Pandas `pivot()` method reshapes the data frame from long to wide.
- In other words, it reduces the number of rows and increases the number of columns.
- Pivoting turns unique values into separate columns.
- Although it violates the tidy data principle (rows containing observations), it reshapes the data frame into a reporting friendly format.

Data Cleaning

Tidy Data

- As you can see, `pivot()` operates on the defined column and carry along the values based on the configured `values` argument.

```
In [32]: df.pivot(index='name', columns='treatment', values='reading')
```

Out[32]:

treatment name	treatment a	treatment b
Aisyah	20	25
Fatimah	16	31
Melur	10	7

Data Cleaning

Tidy Data

- However, `pivot()` only works on columns having unique values. What if Aisyah has two entries of treatment a?
- Pandas provides another method, `pivot_table()` to handle duplicate entries.
- `pivot_table()` has a parameter `aggfunc()` that handles the duplicate entries. By default, `pivot_table()` will take the average value among all the duplicate entries.

Data Cleaning

Tidy Data

- You can specify other functions aside from the default through the `aggfunc` argument.

```
In [64]: df.pivot_table(index='name', columns='treatment', values='reading', aggfunc=np.mean)
```

Out[64]:

treatment	treatment a	treatment b
name		
Aisyah	15.5	25.0
Fatimah	16.0	31.0
Melur	10.0	7.0

Data Cleaning

Sampling Data

- Pandas provide the `sample()` method for you to perform a random sampling of items from your data frame.
- Sampling is useful if you have a real big dataset and wants to analyse a portion of it for generalization.
- The `frac` argument specifies the fraction of data to sample while `n` defines the exact number of data to sample.

```
In [549]: rand
```

Out[549]:

	A	B
0	47	27
1	88	70
2	95	54
3	73	17
4	27	87
5	52	81
6	9	67
7	17	98
8	44	10
9	27	31

```
In [551]: rand.sample(n=3)
```

Out[551]:

	A	B
8	44	10
0	47	27
7	17	98

```
In [550]: rand.sample(frac=0.2)
```

Out[550]:

	A	B
6	9	67
5	52	81

Data Cleaning

Converting Data Types

- You can check the data types of each column in a data frame by using the `dtypes` attribute.
- If you wish to convert the data type, Python provides a method `astype()`.
- For example, if you are converting the ‘treatment’ column from int to a string:
  - `df[‘treatment’] = df[‘treatment’].astype(‘str’)`
- You should convert categorical data into ‘category’ type.
- It reduces the memory footprint and allows wider compatibility with Python libraries.

Data Cleaning

Removing erratic entries

- It is common to have strings in numeric columns. For example, a dash '-' is typical entry for empty rows/columns.
- These dashes are considered unclean data as they negatively affect the analysis process.
- The `coerce` argument in Pandas `to_numeric()` method will force these erratic entries to be NaN.

```
Out[131]:
```

treatment name	treatment a	treatment b
Aisyah	-	25
Fatimah	16	31
Melur	10	7

```
In [132]: df['treatment a'] = pd.to_numeric(df['treatment a'], errors='coerce')

In [133]: df

Out[133]:
```

treatment name	treatment a	treatment b
Aisyah	NaN	25
Fatimah	16.0	31
Melur	10.0	7

Data Cleaning

Duplicate Data

- Duplicate data can skew results.
- It gives false assumption and may lead to wrong decision making.
- To check for duplicates, you can use duplicated() method on your data frame.
- Then, use `drop_duplicates()` method to remove all duplicate rows.
- By default, it drops all duplicates except for the first occurrence.

Data Cleaning

Duplicate Data

```
In [187]: df
```

Out[187]:

	name	treatment a	treatment b
0	Aisyah	20.0	25
1	Fatimah	16.0	31
2	Melur	10.0	7
3	Aisyah	20.0	25
4	Aisyah	20.0	25

```
In [188]: df.duplicated()

Out[188]: 0     False
          1     False
          2     False
          3      True
          4      True
          dtype: bool
```

```
In [189]: df.drop_duplicates()
```

Out[189]:

	name	treatment a	treatment b
0	Aisyah	20.0	25
1	Fatimah	16.0	31
2	Melur	10.0	7

Data Cleaning

Missing Data

- Missing data is one of the most frequent problems faced by data scientists.
- Missing data must be treated accordingly to ensure effective analysis.
- There are 3 common approaches to overcome this problem:
  - Leave as-is (if it does not affect analysis e.g. missing data in unused columns)
  - Drop them (pay attention to the amount of data affected)
  - Fill missing value (using summary statistics e.g. mean, median)
- An easy way to locate missing data is by using `info()` method and look for unequal counts across all columns.

Data Cleaning

Missing Data

- You can drop missing data using the `dropna()` method on your data frame.

```
In [195]: data.info()

          <class 'pandas.core.frame.DataFrame'>
          RangeIndex: 163 entries, 0 to 162
          Data columns (total 5 columns):
          Country            163 non-null object
          Continent          163 non-null object
          female literacy    163 non-null float64
          fertility          163 non-null float64
          population         148 non-null float64
          dtypes: float64(3), object(2)
          memory usage: 6.4+ KB
```

```
In [198]: data = data.dropna()

In [199]: data.info()

          <class 'pandas.core.frame.DataFrame'>
          Int64Index: 148 entries, 0 to 162
          Data columns (total 5 columns):
          Country            148 non-null object
          Continent          148 non-null object
          female literacy    148 non-null float64
          fertility          148 non-null float64
          population         148 non-null float64
          dtypes: float64(3), object(2)
          memory usage: 6.9+ KB
```

Data Cleaning

Missing Data

- As an alternative, you can opt to fill the missing data with statistical values.
- Some common summary statistics are median and mean.
- Median is a more appropriate choice if your data has outliers.
- To display rows with missing values, use the `isnull()` and `any()` method.

```
In [210]: data[data.isnull().any(axis=1)]
```

Out[210]:

	Country	Continent	female literacy	fertility	population
4	Brésil	LAT	90.2	1.827	NaN
16	Turquie	ASI	81.3	2.100	NaN
17	Iran, République islamique d'	ASI	77.2	1.781	NaN
19	Rép. Démocratique du Congo	AF	56.1	5.908	NaN
27	Espagne	EUR	96.9	1.468	NaN
49	Mozambique	AF	40.1	4.967	NaN
51	Côte d'Ivoire	AF	44.3	4.514	NaN
86	Burundi	AF	59.9	4.450	NaN
94	Togo	AF	53.7	4.166	NaN
100	Nicaragua	LAT	77.9	2.683	NaN
145	Chypre	EUR	96.7	1.516	NaN

Data Cleaning

Missing Data

- Use the `fillna()` method and the summary statistic as its argument.
- For example, if you want to fill the missing values with the median, use the `median()` method on your column inside `fillna()`.

```
In [221]: data['population'] = data['population'].fillna(data['population'].median())

In [222]: data.info()

          <class 'pandas.core.frame.DataFrame'>
          RangeIndex: 163 entries, 0 to 162
          Data columns (total 5 columns):
          Country            163 non-null object
          Continent          163 non-null object
          female literacy    163 non-null float64
          fertility          163 non-null float64
          population         163 non-null float64
          dtypes: float64(3), object(2)
          memory usage: 6.4+ KB
```

Data Cleaning

String manipulation

- String manipulation is also an essential part of data cleaning.
- Python has rich string manipulation features. Among them:
  - `str.split()` : split words into Python list based on defined delimiters.
  - `str.replace()` : replace current word with new word.
  - `str.title()` : capitalize first letter of each word in string.
  - `str.capitalize()` : capitalize first letter of the first word in string.
  - `str.strip()` : remove white space from both ends
  - `str.lstrip()` / `rstrip()` : remove white space from left / right.

Data Cleaning

Using Regex

- Regex or regular expression allow granular pattern matching in strings.
- It is really useful to search for a specific pattern or sequence of characters in a string for further processing.
- Python provides the `re` library for regex.
- Using regex gets really confusing at first. It takes time and practice to master the art of using regex.
- Can test your regex here: https://regexr.com

Data Cleaning

Using Regex

- Example match:

17	12345678901	\d*
$17	$12345678901	\$\d*
$17.00	$12345678901.42	\$\d* \.\d*
$17.89	$12345678901.24	\$\d*\.\d{2}
~~$17.895~~	~~$12345678901.999~~	^\$\d*\.\d{2}$

Data Cleaning

Using Regex

- To use regex in Python, you need to do the following:
  1. compile the pattern
  2. use the compiled pattern to match values

```
In [245]: import re

In [246]: pattern = re.compile('[A-Z]{2}\d{6}')

In [247]: result = pattern.match('FA156481')

In [248]: bool(result)

Out[248]: True
```